

Livrable1 - BDD

Batist AUREILLE - INFO 4A FISA

Phase 1 : Exploration et base SQLite

1 Introduction et Plan de la phase 1

A travers cette première phase, nous avons appréhendé le projet Cineexplorer, avec sa base de données IMDB, via Jupyter et réalisé son déploiement avec un schéma relationnel normalisé 3NF. Nous avons par la suite effectué des requêtes complexes et ajouté des index afin de comparer les performances d'exécution.

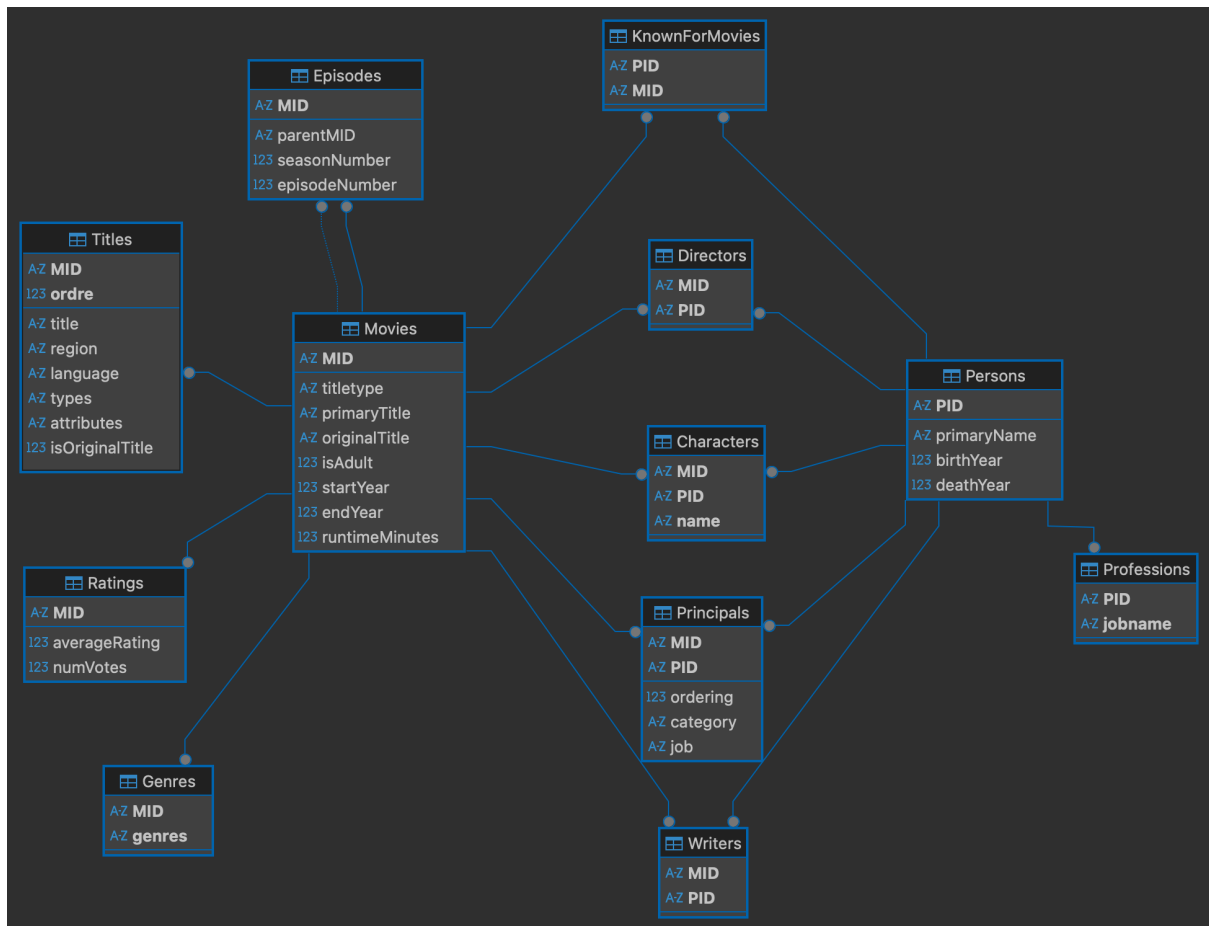
2 T1.0 : Exploration de la donnée

Le fichier Jupyter fourni permet de charger les données des csv fournis et d'effectuer les première analyses sur les données. En plus des histogrammes permettant de mieux comprendre la nature des données présentes, cette partie a surtout mis en lumière la présence de données dites orphelines, c'est-à-dire n'ayant pas d'identifiant dans la table parent, ce qui pose un problème lors de la normalisation, il faudra alors supprimer ces lignes par la suite pour garantir l'intégrité de notre base. A noter l'utilisation d'un LLM afin de créer un affichage clair et agréable à lire pour les résultats.

3 T1.1 : Conception du schéma relationnel et T1.2 : Import des données

L'objectif était de réaliser un schéma relationnel normalisé en forme 3NF de cette base afin d'assurer l'intégrité des données en évitant notamment la redondance. Ci-dessous le schéma correspond permettant de valider ces points clés dans la poursuite de notre projet. Les 2 tables parents sont Movies et Persons, autour desquelles s'articulent l'ensemble des autres tables. Le logiciel DBeaver nous a permis de réaliser ce schéma de manière claire et détaillée en renseignant les clés primaires et étrangères (PK,FK).

Nous avons ensuite implémenté le fichier "create_schema.py" pour créer les tables, en utilisant l'option ON DELETE CASCADE, comme suggéré, pour maintenir l'intégrité lors de suppression de données dans une table parent, et le fichier "import_data.py" afin d'importer les données depuis les fichiers csv en respectant l'ordre d'insertion, en utilisant des transactions et en supprimant les données orphelines des tables enfants. La statistiques du nombre de lignes importées au total est de l'ordre de 12 Millions de lignes pour le jeu de données Medium.



4 T1.3 : Requêtes SQL

Chaque requête réalisée dans le fichier "queries.py" suit le template de requête fourni dans l'énoncé. La réalisation claire du schéma relationnel permet d'aller plus vite sur cette partie car il est alors plus simple d'établir les points de connexion entre les tables pour les différentes jointures. Dans cette partie, nous avons eu recours à des fonctionnalités avancées de SQL comme les multiples jointures, les notions de GROUP BY et HAVING ainsi que RANK.

5 T1.4 : Indexation et benchmark

Cette section du rapport détaille la démarche d'optimisation des performances des requêtes sur la base de données IMDB, en utilisant la notion d'index.

5.1 Objectifs et Méthodologie

L'objectif principal était de quantifier l'impact de l'indexation sur le temps d'exécution d'un jeu de neuf requêtes SQL complexes, représentatives des scénarios d'usage (recherche de filmographie, duos, classements, etc.).

La méthodologie s'est déroulée en trois phases principales :

1. **Etape Recherche** : Mesure du temps d'exécution (sans index) et analyse du plan d'exécution (EXPLAIN) pour établir la performance de référence.

2. **Etape Optimisation** : Création d'index pertinents (voir "banchmark.py") sur les colonnes cibles (WHERE, JOIN, ORDER BY) et exécution de la commande ANALYZE pour mettre à jour les statistiques de l'optimiseur.
3. **Etape Validation** : Nouvelle mesure du temps d'exécution (avec index) et calcul du gain de performance ainsi que de la surcharge de stockage.

5.2 Analyse du Plan d'Exécution (EXPLAIN QUERY PLAN)

L'analyse qualitative, réalisée via la commande EXPLAIN QUERY PLAN sur la requête critique query_actor_filmography a démontré la transformation du plan d'exécution :

- **Avant indexation** : Le plan indiquait un **SCAN TABLE** coûteux sur la table **Persons** (analyse complète de la table), typique des recherches lentes.
- **Après indexation** : Le plan a basculé vers un **SEARCH TABLE ... USING INDEX** (utilisation de l'index `idx_person_name`), confirmant que l'optimiseur utilisait désormais le chemin d'accès rapide créé.

5.3 Utilité Synthétique des Index Créés

L'efficacité des requêtes est assurée par la présence des index suivants, chacun ciblant des opérations spécifiques (WHERE, JOIN, ORDER BY) pour éviter les analyses complètes de tables (*Full Table Scans*).

`idx_person_name` Sur `Persons(primaryName)`. Accélère la recherche d'acteurs ou de réalisateurs par le début de leur nom (LIKE 'Nom%').

`idx_movie_year` Sur `Movies(startYear)`. Optimise le tri et le filtrage des films basé sur l'année de sortie.

`idx_rating_score` Sur `Ratings(averageRating)`. Permet le tri rapide des films par leur note moyenne, crucial pour les requêtes de classement.

`idx_genre_name` Sur `Genres(genres)`. Assure un filtrage rapide par genre (WHERE genres = ?).

`idx_char_pid` / `idx_char_mid` Sur `Characters(PID)` et `Characters(MID)`. Optimisent les jointures de la table des rôles vers les acteurs et vers les films.

`idx_principals_mid` / `idx_principals_pid` Sur `Principals(MID)` et `Principals(PID)`. Optimisent les jointures de la table des rôles principaux (acteurs/actrices) vers les films et vers les personnes.

`idx_directors_mid` / `idx_directors_pid` Sur `Directors(MID)` et `Directors(PID)`. Accélèrent les jointures pour relier les réalisateurs aux films.

`idx_genre_mid_genre` **Index Composé** sur `Genres(MID, genres)`. Améliore simultanément les jointures vers la table `Movies` et le filtrage par genre.

`idx_rating_mid_rating` **Index Composé** sur `Ratings(MID, averageRating)`. Optimise les jointures vers `Movies` tout en préparant le tri par note.

5.4 Résultats et Gains de Performance

Le tableau ci-dessous illustre un exemple de résultats sur les temps d'exécution pour chaque requête, mettant en évidence les gains obtenus. Le gain de performance (Gain(%)) est calculé selon la formule :

$$\text{Gain(\%)} = \frac{(\text{Temps_Sans} - \text{Temps_Avec})}{\text{Temps_Sans}} \times 100$$

Table 1: Comparaison des temps d'exécution (Sans Index vs. Avec Index)

Requête	Sans Index (ms)	Avec Index (ms)	Gain de Performance (%)
query_actor_filmography	9336.21	5250.58	43.76%
query_director_actor_collaborations	1349.9	78.28	94.2%
query_top_n_movies_by_genre	7034.7	5419.72	22.96%
query_multi_role_actors	3703.17	54.91	98.52%
query_popular_genres	1788.05	315.81	82.34%
query_actor_career_evolution	7490.3	5122.73	31.61%
query_top_movies_per_genre_ranked	170.96	178.34	-4.32%
query_breakout_roles	3741.61	2868.47	23.34%
query_golden_duos	18412.81	11034.97	40.07%

5.5 Bilan de l'Optimisation

L'implémentation des index a permis des gains de performance notables sur la quasi totalité des requêtes.

- **Stockage (Surcharge) :** L'ajout des structures d'index a engendré une augmentation de la taille du fichier de base de données de $+ [23.24]$ Mo.

Cette augmentation de la surcharge de stockage est nécessaire pour stocker la structure des index et ainsi gagner en performance temporelle pour les requêtes.

6 Conclusion de la Phase 1

La Phase I a été un succès, validant une base de données SQLite normalisée (3NF) avec des requêtes SQL avancées et des gains de performance significatifs grâce à une indexation ciblée. Cependant, cette structure relationnelle introduit une complexité (multiples JOIN) et une rigidité qui nuisent à l'efficacité des requêtes. De plus, l'optimisation a engendré une surcharge de stockage.

La Phase II (Migration MongoDB) quant à elle, permettra d'exploiter la flexibilité du modèle NoSQL et simplifier les requêtes de récupération d'informations détaillées via la dénormalisation.